



**REPUBLIC OF YEMEN
UNIVERSITY OF ADEN
FACULTY OF ENGINEERING**



Logic Design

Prepared by:

Qassem Tawheed Qassem Saleh

Reg.No:2505071028

Supervisor:

Ms:Aqdar

Part 1: Truth Tables & Boolean Simplification

Problem 1 :- $F(A, B, C) = \Sigma m(0, 2, 4, 6)$

Step 1 — Truth Table

m	A	B	C	F
0	0	0	0	1
1	0	0	1	0
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	0

Step 2 — Simplified SOP (sum of the minterms where $F = 1$, i.e. m_0, m_2, m_4, m_6):

$$F = A'B'C' + A'BC' + AB'C' + ABC'$$

Step 3 — Simplification. Factor out C' from every term:

$$\begin{aligned} F &= C' \cdot (A'B' + A'B + AB' + AB) \\ A'B' + A'B &= A'(B' + B) = A' \cdot 1 = A' \\ AB' + AB &= A(B' + B) = A \cdot 1 = A \\ F &= C' \cdot (A' + A) = C' \cdot 1 = C' \\ F &= C' \end{aligned}$$

Interpretation: the truth table shows $F = 1$ exactly when $C = 0$, regardless of A and B , and $F = 0$ whenever $C = 1$. So the function depends only on C , and the fully minimised form is $F = C'$ — the whole circuit reduces to a single NOT gate.

Problem 2:- $Y = A.B' + A.B + A'.B$

Step 1. Truth Table

AB'	AB	A'B	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	1
1	0	0	1	0	1
0	1	0	1	1	1

The table shows $Y = 0$ only when $A = 0$ and $B = 0$ together - exactly the behaviour of an OR gate, so we expect the simplified result $Y = A + B$. We now prove this algebraically.

Step 2. Algebraic proof using Boolean theorems

$$\begin{aligned} Y &= A.B' + A.B + A'.B \\ &= A.(B' + B) + A'.B \text{ [distribution]} \\ &= A.(1) + A'.B \text{ [} B' + B = 1 \text{]} \\ &= A + A'.B \\ &= A + B \text{ [absorption law: } X + X'.Y = X + Y \text{]} \end{aligned}$$

$\therefore Y = A + B$ (matches the truth table exactly)

Part 2: Logic Gates & Circuit Design

Problem 3 :- $F = (X + Y') . (W' + Z) . X$

The circuit for this expression consists of:

- Two NOT gates to generate Y' and W'
- Two OR gates to generate $(X+Y')$ and $(W'+Z)$
- One 3-input AND gate to combine all three terms (including X directly).

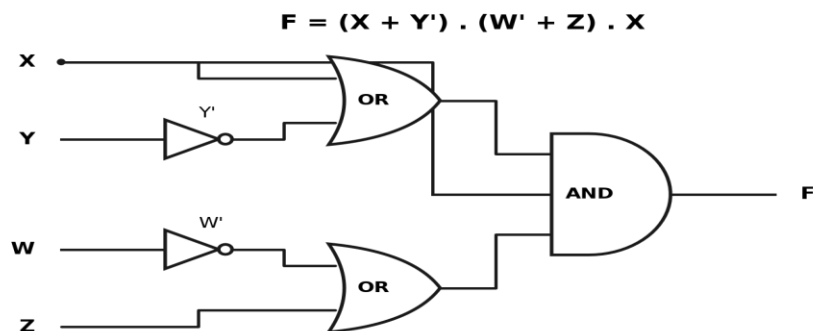


Figure 3: Logic circuit for $F = (X + Y').(W' + Z).X$

Problem 4 — Alarm circuit — three sensors A, B, C

Required condition: "Sensor A must be active, AND simultaneously, either Sensor B is inactive OR Sensor C is active." Translating this directly into a Boolean expression:

$$F = A \cdot (B' + C)$$

Circuit: a NOT gate generates B' , an OR gate combines B' with C , and finally an AND gate combines that result with A :

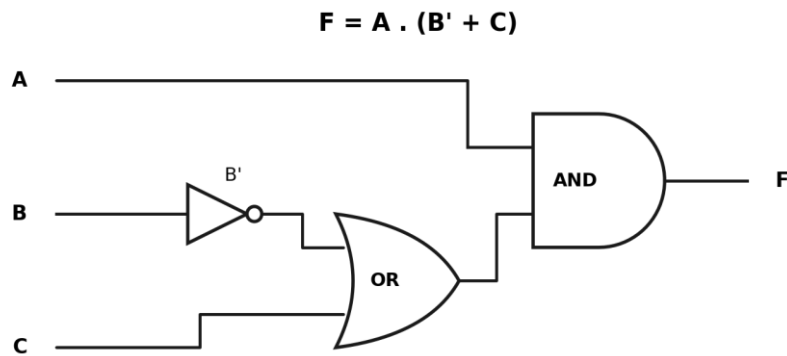


Figure 4: Alarm circuit $F = A \cdot (B' + C)$

Part 3: Karnaugh Maps (K-Maps)

Problem 5 :- $F(X, Y, Z) = \sum m(1, 3, 5, 7)$ — 3-variable K-Map

Place 1s in the cells for minterms m1, m3, m5, m7 on a 3-variable K-map, then find the largest possible grouping (a power of 2):

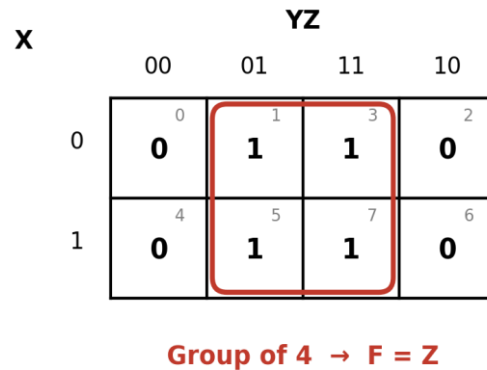


Figure 5: K-Map for Problem 5

All the 1-cells lie in the two columns where $Z = 1$ (covering both rows $X = 0$ and $X = 1$ completely) — a group of 4 adjacent cells. The only variable that stays constant across this group is $Z = 1$, while X and Y vary freely. Therefore:

$$F = Z$$

This means the function needs no logic gate at all to implement — it is simply a direct wire (buffer) from input Z to output F .

Problem 6 :- $F(A, B, C, D) = \sum m(0,1,2,4,5,6,8,9,12,13,14)$ — 4-variable K-Map

Draw a 4-variable K-map (AB as rows, CD as columns in Gray code), then fill in the given minterms:

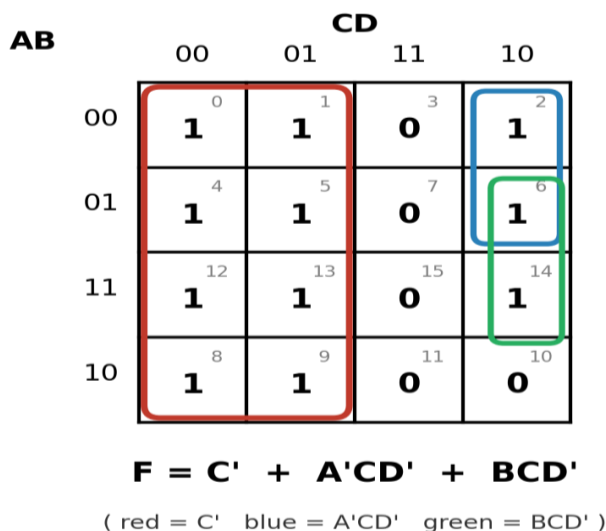


Figure 6: 4-variable K-Map with the three groupings

Optimal groupings:

1) Red group: the entire columns $CD=00$ and $CD=01$ (all four rows) = 8 adjacent cells. The only constant variable is $C = 0 \rightarrow$ term C' .

2) Blue group: two adjacent cells at $AB=00$ and $AB=01$ with $CD=10$. Constant variables: $A=0$, $C=1$, $D=0 \rightarrow$ term $A'CD'$.

3) Green group: two adjacent cells at $AB=01$ and $AB=11$ with $CD=10$. Constant variables: $B=1$, $C=1$, $D=0 \rightarrow$ term BCD' .

Adding the three group terms gives the final minimised SOP expression:

$$F = C' + A'CD' + BCD'$$

Part 4: The Half Adder

Problem 7 :- Design a Half Adder from scratch

Step 1 — Truth table for adding two bits A and B

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Step 2 — Boolean equations

$$\text{Sum} = A \oplus B = A.B' + A'.B$$
$$\text{Carry} = A . B$$

Step 3 — Logic circuit (one XOR gate and one AND gate):

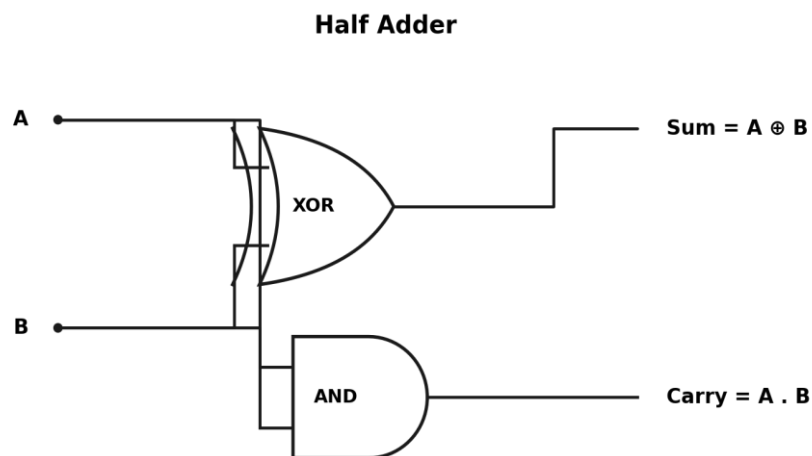


Figure 7: Half Adder circuit

Problem 8 :- Half Adder using NAND gates only

A half adder can be built entirely from NAND gates (as a universal gate): four NAND gates generate the Sum signal, and a fifth NAND gate (wired as an inverter) generates the Carry signal:

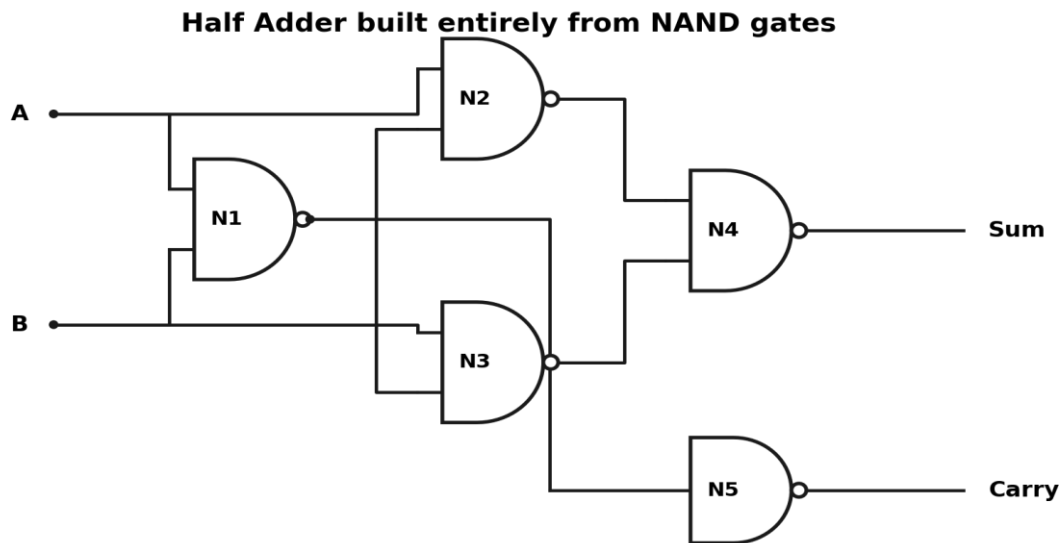


Figure 8: Half Adder built entirely from NAND gates

Algebraic proof of correctness:

$$\begin{aligned} N1 &= (A.B)' \\ N2 &= (A.N1)' = A' + N1' = A' + A.B = A' + B \\ N3 &= (B.N1)' = B' + N1' = B' + A.B = B' + A \\ \text{Sum} &= (N2.N3)' = N2' + N3' = A.B' + A'.B = A \oplus B \quad \checkmark \\ \text{Carry} &= (N1.N1)' = N1' = A.B \quad \checkmark \end{aligned}$$

Part 5: The Full Adder

Problem 9:- Full Adder — derive Cout

Step 1 — Truth table for the Full Adder (three inputs: A, B, Cin)

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Step 2 — Deriving Cout. Cout = 1 for minterms m3, m5, m6, m7:

$$\begin{aligned} \text{Cout} &= A'.B.C \\ &+ A.B'.Cin + A.B.Cin' + A.B.Cin \end{aligned}$$

Group the last two terms, which share A.B:

$$\begin{aligned} A.B.Cin' + A.B.Cin &= A.B.(Cin' + Cin) = A.B \\ A'.B.Cin + A.B'.Cin &= Cin.(A'.B + A.B') = Cin.(A \oplus B) \\ \text{Cout} &= A.B + Cin.(A \oplus B) \end{aligned}$$

This matches the required standard form exactly.

Problem 10 :- Full Adder from Two Half Adders + OR gate

We use two half adders (HA1, HA2) and a single OR gate, tracing the internal signals as follows:

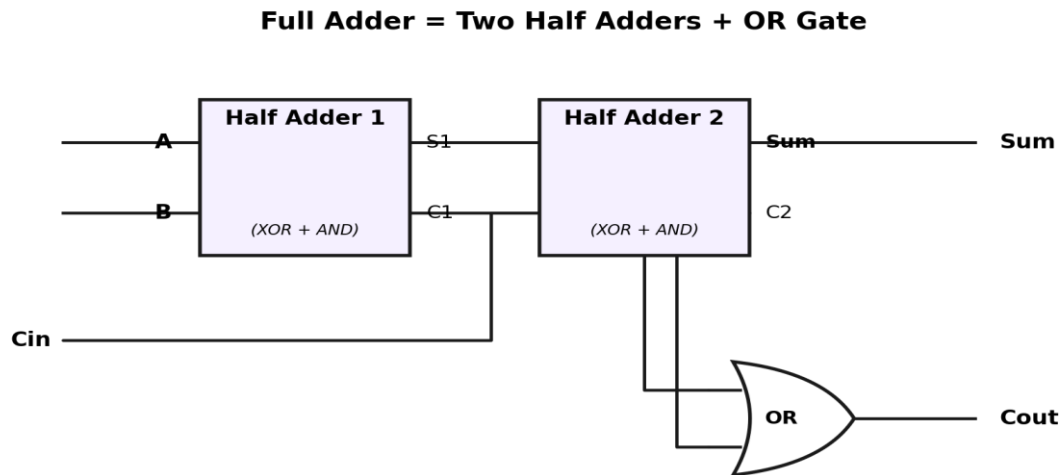


Figure 10: Building a Full Adder from two Half Adders and an OR gate

- Half Adder 1 (HA1): inputs A and B, outputs $S1 = A \oplus B$ and $C1 = A.B$
- Half Adder 2 (HA2): inputs S1 and Cin, outputs the final $Sum = S1 \oplus Cin = A \oplus B \oplus Cin$, and an internal carry $C2 = S1.Cin$
- The OR gate combines the two carry signals from both half adders to give the final carry-out:

$$Cout = C1 + C2 = A.B + Cin.(A \oplus B)$$

This result exactly matches what was derived in Problem 9, confirming the design is correct.